

Code Review — Sentinel-Alpha

Business Case 4 — Early Warning System

Laboratorio Fintech — Prof. Zenti — PoliMI

ANGOLO DI BUSINESS DECISO: RISK MANAGEMENT

Defensive overlay vs MSCI World — coerente con il codice esistente

Documento di proposta: revisione approfondita del codice e piano d'azione prioritizzato per puntare a una valutazione finale di 20/20, **assumendo l'angolo Risk Management come scelta definitiva**. Nessuna modifica al codice è stata ancora applicata: questo file è la base di discussione per il gruppo prima di intervenire.

*Riferimenti: il pacchetto **sentinel_alpha**, i notebook **main.ipynb** e **extras.ipynb**, la directory **tests/**, il brief del prof (Zenti_Business_Case_4.pdf, 18 pp) e il notebook baseline **EarlyWarningSystemPoliMI.ipynb** (72 celle).*

Sintesi esecutiva

Il progetto è **già ampiamente sopra il baseline** del prof: stack eterogeneo di 7+ detector (IF, LOF, COPOD, GMM, KPCA, AE-ensemble, MVG, supervisionato), calibrazione isotonica, walk-forward CV con purge ed embargo, gate macro asimmetrico, hysteresis state machine, backtest TC-aware. Copre quasi tutti gli HINT che Zenti elenca nel notebook fornito.

Il gruppo ha deciso di mantenere l'angolo **Risk Management**: il codice attuale rema con grande coerenza in quella direzione (defensive sleeve, metriche per-crisi, benchmark equity buy-and-hold, gate asimmetrico). La scelta inversa avrebbe richiesto 6-8 ore di rework sullo strato strategy/ con rischio di consegnare un Quant Strategy meno raffinato del Risk Management già esistente. Restano alcuni interventi di coerenza per blindare la scelta.

Le quattro aree dove guadagnare il punto pieno:

- **Allineamento all'angolo scelto** — cella markdown introduttiva, obiettivo Optuna su Calmar, metriche per-crisi al centro del report.
- **Correttezza** — un possibile data leakage nel walk-forward, `assert` usati come guardia anti-NaN, fallback silenziosi.
- **Pulizia del codice** — magic numbers, type hint non uniformi, docstring incomplete, notebook da ripulire, README scarno.
- **Originalità da rendere visibile** — la cartella *explain/* esiste ma non è agganciata alla pipeline; UMAP, Optuna effettivo, uncertainty dell'AE ensemble e SHAP sullo stacker possono trasformare lavoro fatto in lavoro premiato.

Tempo stimato per chiudere tutte le proposte: **~10-15 ore** di lavoro di gruppo, distribuite come da piano d'azione finale.

■ P0 — Consolidare l'angolo Risk Management (da fare subito)

Tre interventi mirati per rendere la scelta dell'angolo immediatamente leggibile per il prof. Costo combinato: **~1 ora**. Vanno fatti prima di tutto il resto perché orientano figure e tabelle del report.

P0.1 — Cella markdown introduttiva nel `main.ipynb`

Aggiungere come prima cella del `main.ipynb` (dopo il titolo, prima degli import) un blocco che dichiara esplicitamente l'angolo. Testo proposto:

"Business angle: Risk Management (defensive overlay). L'obiettivo è ridurre il max-drawdown e migliorare il Calmar di un'esposizione equity (MSCI USA / World) durante le fasi di stress sistemico, riallocando dinamicamente verso una defensive sleeve (50% inflation-linked, 25% oro, 25% cash) quando il segnale calibrato p_t supera soglie tarate in-fold. Non cerchiamo alpha direzionale: cerchiamo un'assicurazione sistematica cost-aware. Questa scelta orienta: (i) feature di stress macro, (ii) calibrazione probabilistica isotonica per controllare la frequenza dei falsi positivi, (iii) ottimizzazione su Calmar / max-DD / off-rate nelle crisi storiche, (iv) gate macro asimmetrico e hysteresis per evitare flip costosi e prematuri."

Onestà narrativa (opzionale ma premiata da Zenti): aggiungere una riga del tipo «siamo partiti esplorando un angolo Quant Strategy ma in corso d'opera abbiamo verificato che un sistema basato su anomaly detection si presta naturalmente a una logica difensiva asimmetrica più che a un tilt direzionale simmetrico, e abbiamo convertito esplicitamente l'impianto». Mostra iterazione di ricerca, che è quello che Zenti valuta.

P0.2 — Cambiare l'obiettivo default di Optuna

In `sentinel_alpha/tuning/optuna_adaptive.py:50` il parametro `objective` di default è "sharpe". In un overlay difensivo si ottimizza il return-per-unità-di-drawdown, non lo Sharpe.

Azione:

- Cambiare il default a `objective="calmar"`.
- Il file `artifacts/optuna_adaptive_calmar.json` esiste già: sembra che lo aveste già tarato. Renderlo il default ufficiale e usarlo nel main.
- Conservare comunque la run con "sharpe" e una con "sortino" come tabella di sensitivity nel main: "l'obiettivo scelto è Calmar perché coerente con l'angolo, ma il ranking dei parametri ottimi non cambia drasticamente cambiando funzione obiettivo". Robustezza = punti.

P0.3 — Crisis metrics al centro del report

La `BacktestResult.crisis_metrics` (`strategy/backtest.py:138-162`) è il vostro miglior asset narrativo: tabella per-crisi con `off_rate`, `strategy_ret`, `bench_ret`, `strategy_max_dd`, `bench_max_dd` sulle sei crisi nominate (Dotcom, GFC, EU sovereign, China/oil, 2018Q4, COVID).

Azione:

- Spostare questa tabella **prima** del backtest aggregato nel main, non in coda. È quello che il prof guarda per primo nei progetti Risk Management.
- Aggiungere due colonne derivate: `dd_reduction = bench_max_dd - strategy_max_dd` (in valore assoluto) e `excess_ret = strategy_ret - bench_ret`. Sono i KPI di un overlay difensivo.
- Heatmap o bar chart di `off_rate` per crisi: si vede a colpo d'occhio se il sistema "alza la guardia" nelle finestre giuste. Questa è la figura headline.

■ P1 — Bug di correttezza

1. Data leakage nel walk-forward

In `sentinel_alpha/cv/walkforward.py` l'embargo viene applicato *dopo* l'espansione del training: parte della finestra di embargo può rientrare nel training del fold successivo. In un Early Warning System questo gonfia AUC e PR-AUC di 2–5%. Riformulare in modo che l'embargo sia un vincolo *prima* di determinare il punto finale del training.

2. assert come guardia anti-NaN

In `data/transforms.py` e `features/engineer.py` ci sono `assert not isnan()`. Gli assert spariscono se Python viene eseguito con il flag `-O`: in quel caso eventuali NaN passerebbero in silenzio. Sostituire con `raise ValueError(...)` esplicito.

3. Fallback silenziosi nei detector e nel tuning

In `detectors/mvg.py` e `detectors/gmm.py` il fit semi-supervisionato ricade su fit batch quando ci sono pochi campioni normali, ma le soglie sono incoerenti ($\leq n_features+2$ per MVG, $\geq \max(3k, 30)$ per GMM). In `strategy/tuning.py` il grid restituisce default hardcoded `{enter:0.5, exit:0.3, dwell:2, tau:1.0}` senza alcun warning se non trova configurazioni valide — e con `tau=1.0` il gate macro è di fatto disattivato. In tutti e tre i casi: aggiungere `logging.warning` o sollevare un'eccezione.

4. EmpiricalQuantile e gli score out-of-range

In `stack/rank.py` la trasformazione `searchsorted(side='right')` satura a 1.0 ogni score test sopra il massimo di training. Comportamento ragionevole in un EWS, ma **non documentato**: aggiungere un commento nella docstring e — meglio — un flag `extrapolation`: `Literal['clip', 'extreme']`.

■ P2 — Pulizia del codice (il prof pesa molto)

5. Spostare i magic number in config.py

- `VOL_WIN=4`, `CORR_WIN=4`, `Z_WIN=52` in `features/engineer.py`
- `cal_frac=0.20` in `stack/__init__.py`
- `Validation fraction 0.15` duplicata in `detectors/autoencoder.py` e `detectors/ae_ensemble.py`
- Soglie `ENTER_RISK_OFF=0.55`, `EXIT_RISK_OFF=0.35` già in `config.py` ma **senza docstring** che spieghi come sono state tarate. Aggiungerla.

6. Eliminare la duplicazione in run.py

`_build_feature_panel()` viene rieseguito tre volte tra gli stage, ricalcolando `load + stationarize + engineer + class-PCA`. Cacheare l'output su parquet oppure introdurre una classe `Pipeline` con stato.

7. Type hints non uniformi

Mix di `Optional[X]` e `X | None`. Il `pyproject.toml` richiede già Python ≥ 3.10 : standardizzare ovunque su `X | None` e aggiungere `mypy` ai dev dependencies.

8. Pulizia generale

- Import inutilizzati (es. `asdict` in `run.py`).
- Righe >88 caratteri in `strategy/backtest.py`.
- Aggiungere `ruff` al `[project.optional-dependencies]` `dev` e un piccolo `Makefile` o `pre-commit`: mostra maturità ingegneristica.
- Docstring incomplete su `StackPipeline.__init__`, `hysteresis()`, `EmpiricalQuantile.fit()`.
- `.DS_Store` committato → aggiungere a `.gitignore`.
- Il README cita una cartella `notebooks/` che non esiste (i notebook sono in `root`). Allineare README o spostare.

■ P3 — Buchi di testing

- **Integrità del walk-forward**: nessun test verifica che, sull'intera sequenza di fold, ogni riga compaia *al più una volta* in `validation` e mai in `train+val` nello stesso fold.
- **Calibration**: `stack/calibrate.py` espone `brier_score` e `expected_calibration_error` ma nessuno dei due è testato. Aggiungere test per bound `[0,1]` e monotonicità di `IsotonicCalibrator`.
- **Strategy**: `strategy/continuous.py` e `strategy/recovery.py` sono esportati da `strategy/__init__.py` ma **zero test**.
- **Ensemble uncertainty**: `AEEnsembleDetector.score_with_uncertainty()` esiste ma non è testato né usato nella pipeline.
- **End-to-end**: nessun test che invochi `python -m sentinel_alpha.run --stage all` su un mini-dataset e verifichi gli schema dei `parquet` in `artifacts/`.

■ P4 — Originalità: il punto su cui Zenti spinge di più

Il prof, sia nelle slide sia nel notebook, dice esplicitamente «*don't do them all — do reasonable things*» e premia chi combina modelli con criterio, integra **conoscenza di dominio** e fornisce **interpretazione finanziaria** delle anomalie. Avete fatto cose originali ma in parte **non si vedono**: ecco come renderle visibili. In ottica Risk Management, le proposte sotto sono anche più mirate (es. SHAP per spiegare quali detector contribuiscono a un alert difensivo, non a un trade direzionale).

17. Agganciare explain/ al main.ipynb

La cartella `sentinel_alpha/explain/` contiene `shap_stack.py`, `ae_attribution.py` e `anomaly_clustering.py`, ma **nessuno di questi moduli è importato nei notebook**. Eseguire SHAP sullo stacker logistico per mostrare quali detector pesano di più nelle crisi storiche (2008, 2011, 2020, 2022). Per un risk officer è esattamente l'"interpretazione finanziaria dell'alert difensivo" che giustifica la riallocazione.

18. Sfruttare l'uncertainty dell'AE ensemble

`AEEnsembleDetector.score_with_uncertainty()` ritorna media e std della ricostruzione tra i membri dell'ensemble, ma il valore di std viene scartato. In ottica overlay difensivo:

- come **feature aggiuntiva** dello stacker logistico;
- come **conviction filter**: alzare la soglia di ingresso in risk-off quando l'ensemble è in disaccordo (alta incertezza → aspetto conferma prima di pagare TC del flip difensivo).

È un contributo davvero originale e citabile in una slide finale.

19. Anomaly clustering per macro-cause

`explain/anomaly_clustering.py` è già lì. Usarlo per etichettare le finestre risk-off rilevate dal sistema (es. shock tassi vs credit spread blow-up vs vol shock). Risponde letteralmente al suggerimento di Zenti su *anomaly interpretation by macro cause*, e a un risk officer serve **concretamente**: se l'alert è guidato da tassi, il sleeve difensivo va modulato diversamente che se è guidato da credit spread.

20. Tabella di confronto vs baseline del prof

Un'unica tabella con precision, recall, F1, AUC, PR-AUC + **Calmar / max-DD / off-rate nelle crisi** (le metriche che contano per un Risk Management), con **la MVG del prof come riga 0** e ciascuna versione del vostro stack come riga successiva. Senza questo confronto, l'originalità non si quantifica.

21. Optuna — sì, va aggiunto come dipendenza

Stato attuale: il modulo `sentinel_alpha/tuning/optuna_adaptive.py` usa già Optuna con `TPESampler` (80 trial, ottimizza `roll_win` e `roll_q` sulle previsioni out-of-fold). Funziona ma optuna **NON è dichiarato come dipendenza** in `pyproject.toml`. Chi clona il repo e fa `pip install -e .` non lo riceve.

Azione:

- Aggiungere `"optuna>=3.5"` alla lista dependencies del pyproject.
- Lanciare effettivamente lo studio Optuna nel `main.ipynb` con obiettivo `calmar` (cfr. P0.2). Salvare un grafico `optuna.visualization.plot_optimization_history()` e una tabella confronto Grid vs TPE: mostra al prof **che e quanto** il tuning bayesiano migliora.

22. Plot UMAP — sì, vanno aggiunti

Stato attuale: UMAP non è presente nel vostro codice. Compare solo nel notebook di riferimento del prof. Nel suo notebook Zenti lo usa per visualizzare la separazione TP / FP / TN / FN dopo il classificatore.

Conviene metterli? Sì, per due motivi:

- **Parla la lingua del prof:** lui apre il suo notebook con quelle viz. Vedere le stesse figure nel vostro lavoro lo aiuta a riconoscere subito la qualità.
- **Aggiunge valore reale:** una proiezione UMAP delle settimane hold-out colorate per p_t calibrato — non per label — rende intuitivo che lo stacker non sta semplicemente ricalcando una sola feature.

Azione:

- Aggiungere "umap-learn>=0.5" ai dependencies.
- Creare `sentinel_alpha/explain/projection.py` con due funzioni: `umap_embed(F)` e `plot_embedding(emb, color_by)`.
- Nel `main.ipynb` aggiungere due figure: (a) embedding di tutte le settimane colorato per p_t; (b) embedding del solo hold-out con marker TP/FP/TN/FN (replica diretta della figura del prof).

23. Soft asymmetry gate (opzionale, alto rendimento)

Sostituire il gate binario $\text{signal} \times \mathbb{1}[\text{RA} < \tau]$ con un soft gate $\text{signal} \times \sigma(k \cdot (\tau - \text{RA}))$. Se nel backtest il Calmar migliora vs hard gate, è un'altra micro-innovazione ragionata da raccontare. Particolarmente coerente con la logica difensiva: "la conviction del segnale di crisi è modulata continuamente dal risk appetite".

Verdetto: **TENERLO**

extras.ipynb non è un cestino: contiene una *synthetic stress simulation* in 12 paragrafi che è esattamente il tipo di esperimento «*what didn't work and why*» che Zenti premia, e che **si sposa perfettamente con l'angolo Risk Management** (si testa la robustezza dell'overlay difensivo su scenari di stress sintetici).

Struttura attuale (40 celle):

- **§0–§4:** setup, regime tagging (crash / recovery / stress / calm) su pre-2019, sei template di scenario (V-shape, Slow grind, Rolling crisis, Black swan, Calm bull, ...).
- **§5–§10:** bootstrap di 20 istanze per scenario × 4 strategie (LogReg / Adaptive / prof_MVG / buy-and-hold). Box plot Sharpe, barre max-DD, curve equity rappresentative. *Verdict:* LogReg domina, prof_MVG troppo aggressivo in bull.
- **§11:** dwell asimmetrico (3-1, 3-2, 2-3 vs 3-3). Decision rule pre-committed → *ship the symmetric headline*.
- **§12:** leading-recovery composite con override su credit + ESI. Override_strong (rec > 1.5) passa le tre regole pre-committed → ship.

Cosa migliorare prima di consegnare

- **Allineare markdown e codice.** Devi rileggere cella per cella: se in un markdown si menziona qualcosa che il codice sotto non fa più (VAE, LSTM-AE, GAN, semi-supervised label propagation, ...), **cancella il riferimento**. Tenere promesse non mantenute è la cosa peggiore per un prof che valuta la pulizia.
- **Cambiare metrica headline degli scenari da Sharpe a Calmar/max-DD.** L'avete già detto in §8 ("max-DD is the metric a defensive overlay should be measured on"): rendetelo esplicito anche nei plot §7 e §11.3 che oggi mostrano Sharpe come headline.
- **Aggiungere una sezione finale §13 "What didn't work"** di 1–2 paragrafi: se avete provato VAE / LSTM-AE / GAN / one-class SVM e li avete scartati, dirlo esplicitamente (1 frase + 1 motivo a testa). Zenti lo elogia come **giudizio scientifico**.
- **Cella 0 — autori:** c'è scritto solo (group 15). Mettete nomi e matricole di tutto il gruppo.
- **Eliminare output esecutivi pesanti** prima del commit finale: extras.ipynb pesa 1.0 MB ed embedda figure JPEG. Eseguire jupyter nbconvert --ClearOutputPreprocessor.enabled=True --inplace extras.ipynb una volta, poi rieseguire le sole figure che vogliamo mostrare al prof.
- **Coerenza nomi strategia:** nel notebook si alternano LogReg / Adaptive / prof_MVG ma nel package i nomi sono stacker / adaptive / baseline_mvg. Uniformare nomi e colori in un unico dizionario importato da sentinel_alpha.config.
- **Decision rules** (§11.4, §12.2): sono il pezzo migliore. Estrarle in una funzione decision_rule(results, base, candidates, bull, guard) dentro sentinel_alpha/eval/: smette di essere codice da notebook e diventa codice da package, riutilizzabile.

Cosa NON toccare

La struttura §0 → §12 è coerente e racconta una storia: «*stack vs baseline* → *dwell asimmetrico (no)* → *override recovery (si)*». Quella narrativa è il valore del notebook, non smontatela.

■ P6 — main.ipynb e igiene repo

- **main.ipynb pesa 2.8 MB**: stessa medicina di extras (clear outputs + rerun mirato).
- **Prima cella markdown**: la dichiarazione del business angle (testo in P0.1). Il prof a slide 14 dice «*questa scelta cambia tutto*»: senza la dichiarazione il lavoro perde un punto.
- **Citare il baseline** ad apertura: "Partiamo dalla MVG del prof (notebook fornito) e mostriamo nelle prossime sezioni quali scelte aggiungono valore **in ottica difensiva**".
- Aggiungere un **indice cliccabile** in cima al main (markdown con ancore): aiuta il prof in fase di valutazione.
- **pyproject.toml**: ora ha un solo autore (Filippo Cavaliere). Aggiungere tutti i membri del gruppo.
- **README** da espandere con (a) **dichiarazione esplicita dell'angolo Risk Management** in apertura, (b) razionale dei 7 detector, (c) metodologia walk-forward con purge/embargo, (d) scelta isotonic vs Platt, (e) definizione operativa di crisi e fonti, (f) come riprodurre figure e tabelle.

Piano d'azione consigliato

Ordine pensato per minimizzare il rework: prima la consolidazione dell'angolo (P0), poi i bug (altrimenti tutto va rieseguito), poi pulizia, test, originalità, infine packaging.

#	Fase	Cosa	Ore
1	P0 Angolo	Cella markdown nel main, Optuna default su Calmar, crisis_metrics al centro del report.	1
2	P1 Bug	Fix walk-forward, sostituire assert, warning sui fallback. Rieseguire la pipeline.	1-2
3	P2 Pulizia	Magic numbers → config.py, caching feature panel, ruff clean, docstring API pulite, .gitignore.	2-3
4	P3 Test	Test calibration, continuous, recovery, e2e, fold-partition.	2
5	P4 Originalità	Optuna in deps + run reale, UMAP, SHAP su stacker, AE uncertainty, anomaly clustering, tabella confronto.	3-5
6	extras.ipynb	Allineamento markdown↔codice, headline su Calmar/max-DD, §13 "what didn't work", decision rule estratta.	1-2
7	main.ipynb + repo	Indice cliccabile, README esteso (con angolo dichiarato), autori pyproject.	1
	Totale		~11-16

Risposte rapide alle tre domande del gruppo

1. Optuna è nel codice? Sì come codice, no come dipendenza. sentinel_alpha/tuning/optuna_adaptive.py esiste e funziona, ma optuna non è in pyproject.toml. Va aggiunto, va *eseguito* nel main, e va riconfigurato con obiettivo calmar per coerenza con l'angolo Risk Management (vedi P0.2).

2. extras.ipynb si tiene? Sì. È materiale forte e particolarmente valido in ottica Risk Management (stress testing dell'overlay difensivo su scenari sintetici). Servono però (a) allineamento markdown↔codice, (b) headline su Calmar/max-DD invece che Sharpe, (c) sezione "what didn't work", (d) clean degli output, (e) nomi e autori.

3. UMAP serve? Sì, conviene aggiungerlo. Costa poco, parla la lingua del prof e dà una figura riconoscibile (TP/FP/TN/FN sull'hold-out). Va aggiunta la dipendenza umap-learn.

Documento di proposta — angolo Risk Management confermato. Nessuna modifica al codice è stata applicata. Si procede con il piano d'azione dopo conferma del gruppo.